AWS PROJECT DOCUMENTATION

Load Balanced Architecture with Advanced Request Routing

Path-based and host-based routing with an Application Load Balancer, EC2, S3 and IAM

Created by	Rutuja Naik
Platform	Amazon Web Services (Region: us-east-1)
Services	EC2, Application Load Balancer, Target Groups, S3, IAM, Security Groups
Document type	Step-by-step project / lab documentation

1. Project Overview

This project builds a simple two-page website hosted on Amazon EC2 behind an **Application Load Balancer (ALB)**. Two web servers, **Red** and **Blue**, serve different pages. The ALB inspects each incoming request and forwards it to the correct target group using **advanced request routing** rules.

Path-based routing (also called URL-based routing) forwards a request based on the URL path, for example `/red` or `/blue`. **Host-based routing** forwards a request based on the host header, so a single load balancer can serve several domains by sending each to a different set of EC2 instances or containers.

1.1 Objectives

- Create the web content bucket in S3 and a least-privilege IAM role so EC2 can read it.
- Launch two EC2 web servers (Red and Blue) that bootstrap themselves from S3 using user data.
- Create one target group per server, each with its own health check path.
- Create an internet-facing Application Load Balancer spanning two Availability Zones.
- Configure path-based routing rules (/red* and /blue*) and test them.
- Optionally switch to host-based routing rules.

1.2 Prerequisites

- An AWS Free Tier account.
- Basic familiarity with the AWS Management Console.
- The `advanced-request-routing-code.zip` file (web pages, assets, user data scripts and `bucket-permissions.json`).

1.3 AWS Resources Used

Resource	Name	Purpose
S3 bucket	arr-bucket-xxxxxx (globally unique)	Stores the website files that the instances download
Security group	WebsiteSG	Allows inbound HTTP (TCP 80) from 0.0.0.0/0
IAM policy	S3-ARR-Policy	Grants read access to the S3 bucket
IAM role	S3-ARR-Role	EC2 instance profile that uses S3-ARR-Policy
EC2 instance	Red (us-east-1a)	Apache web server serving /red
EC2 instance	Blue (us-east-1b)	Apache web server serving /blue
Target groups	Red, Blue	Health checks: /red/index.html and /blue/index.html
Load balancer	LabLoadBalancer	Internet-facing ALB, listener HTTP:80, routing rules

2. Architecture Diagrams

The diagrams below show the overall solution and how the load balancer routes requests in each mode.

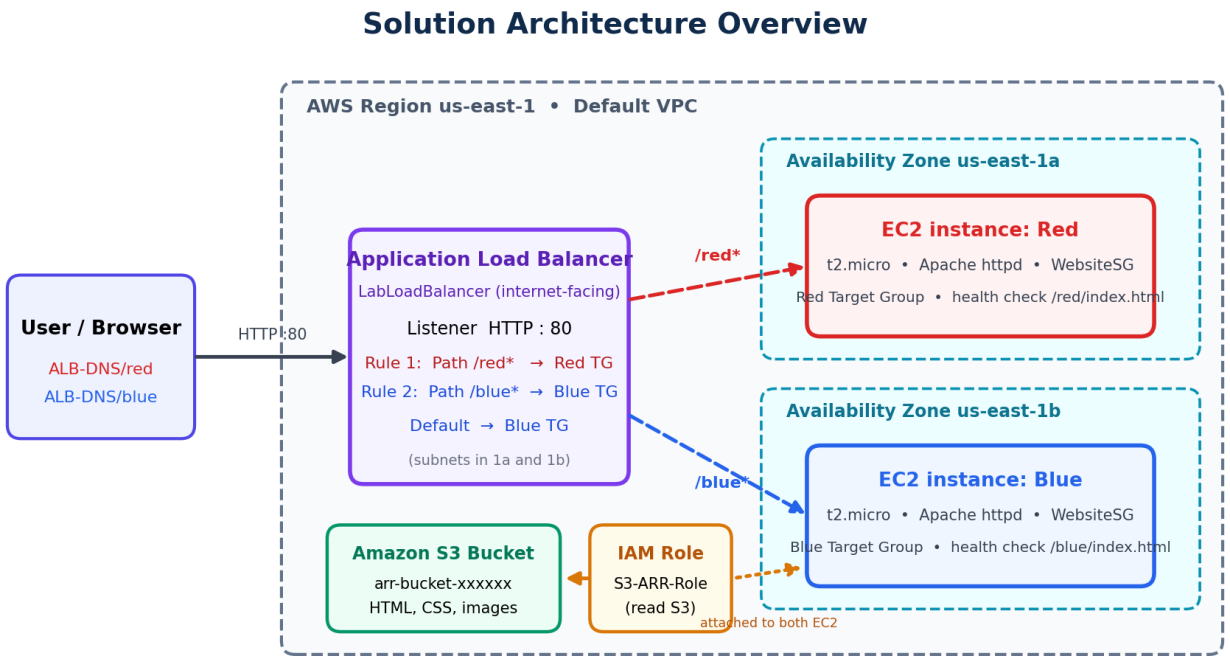


Figure 1: Solution architecture overview (drawn for this document)

2.1 Path-Based Routing

The ALB evaluates the URL path of each request against its listener rules in priority order. A request to `/red` matches rule 1 and goes to the Red target group; `/blue` matches rule 2 and goes to the Blue target group.

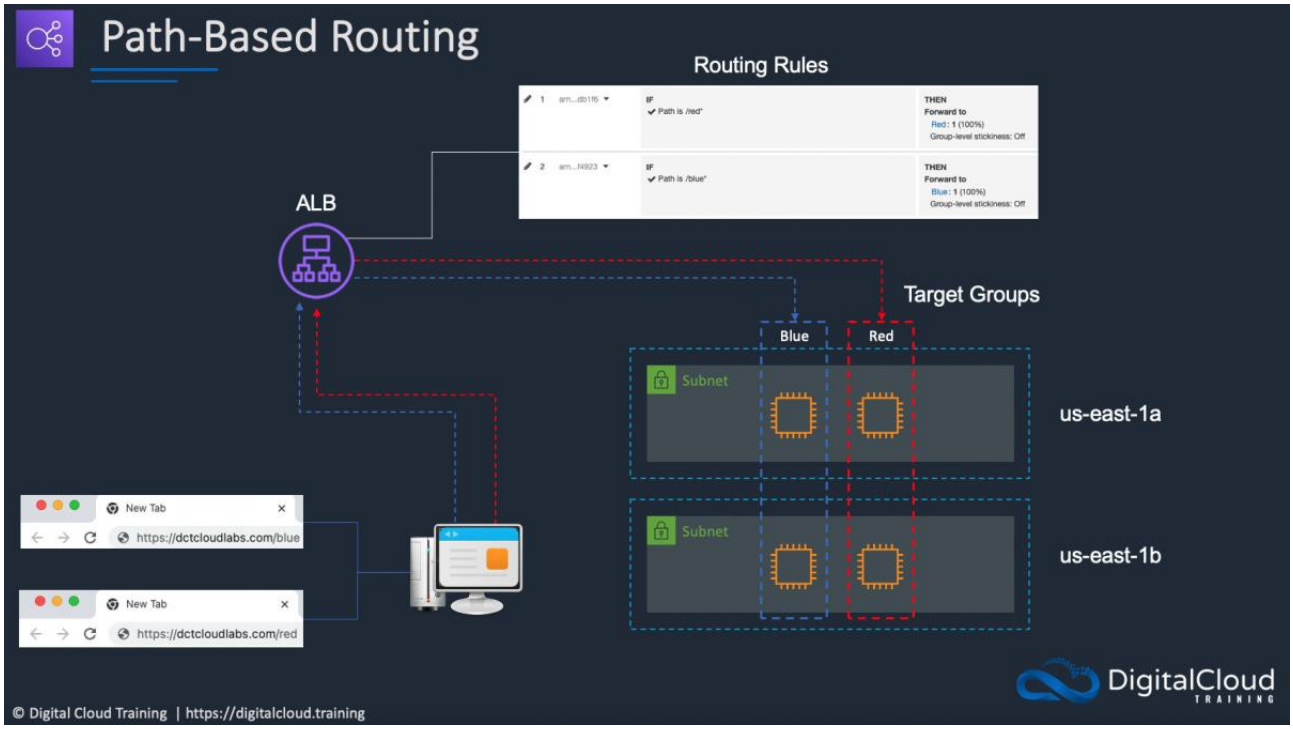


Figure 2: Path-based routing architecture

2.2 Host-Based Routing

With host-based routing the ALB reads the HTTP host header (for example a "red" or "blue" subdomain) and forwards the request to the matching target group. The same load balancer, listener and target groups are reused; only the listener rules change.

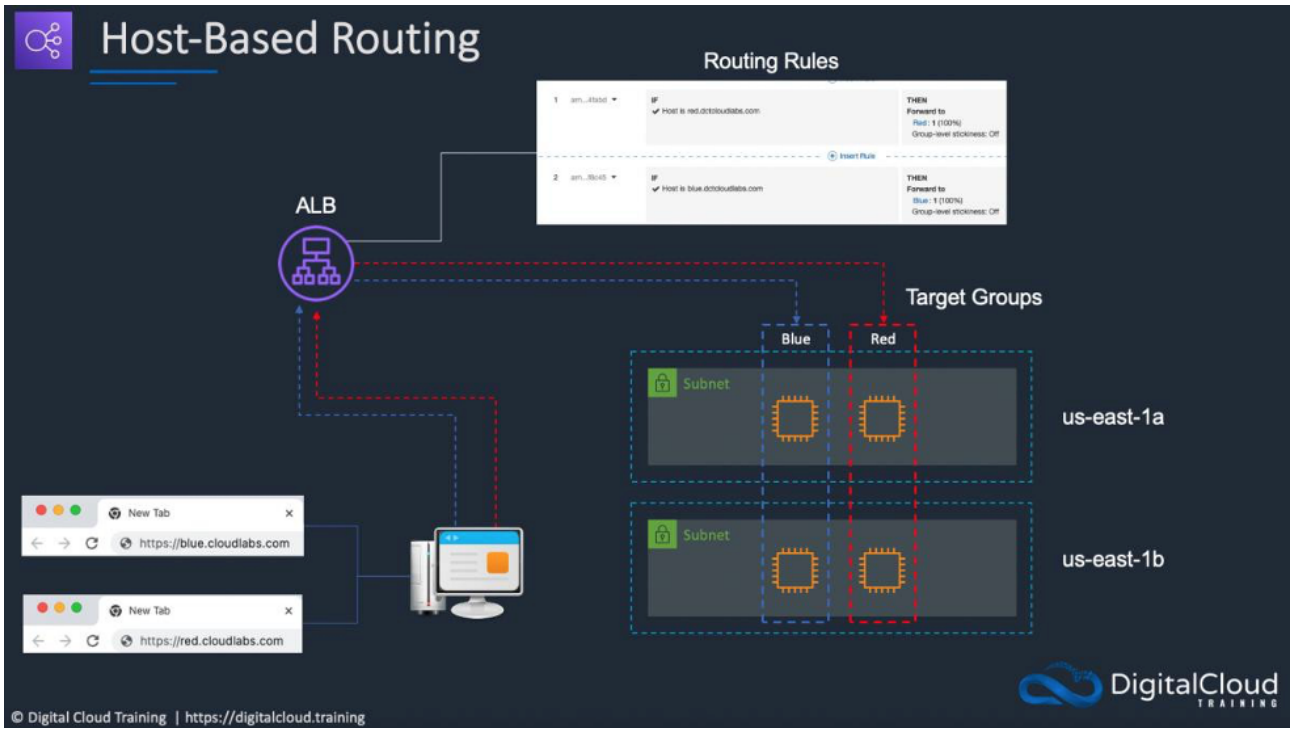


Figure 3: Host-based routing architecture

2.3 Path-Based vs Host-Based Routing

	Path-based	Host-based
Routing key	URL path (/red*, /blue*)	HTTP host header (red.<domain>, blue.<domain>)
Rule condition type	Path	HTTP host header
Typical use	One domain, several apps/sections	Several domains or subdomains on one ALB
Needs extra DNS names	No	Yes, each hostname must resolve to the ALB
Priority in this lab	Red = 1, Blue = 2	Red = 1, Blue = 2

3. Implementation Steps

PHASE 1

Create the S3 Bucket and Upload the Code

Create the bucket

1. Open the **S3 console** and click **Create bucket**.
2. Name the bucket something globally unique, for example `arr-bucket-123456` (the digits are random). **Write the name down**; you will need it again.
3. Scroll down and click **Create bucket**.

Upload the files

1. Download and extract `advanced-request-routing-code.zip`.
2. Upload **all files except** the two user data files and `bucket-permissions.json` to the bucket.

PHASE 2

Create the Security Group and IAM Role

Create the security group (WebsiteSG)

1. In the **EC2 console**, under **Network & Security**, open **Security Groups** and click **Create security group**.
2. Name: `WebsiteSG`. Description: `WebsiteSG`.
3. Add one inbound rule: Type **HTTP**, Protocol **TCP**, Port range **80**, Source **Anywhere-IPv4 (0.0.0.0/0)**.
4. Click **Create security group**.

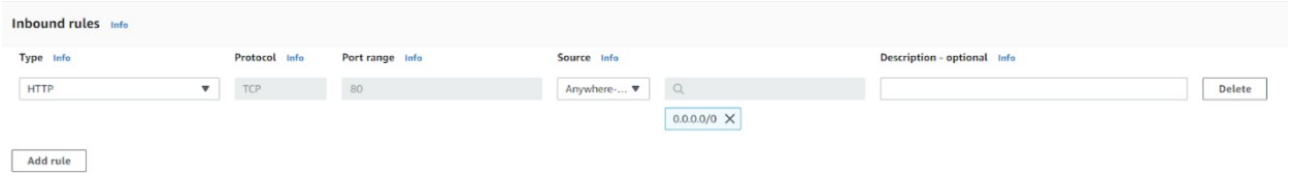


Figure 4: WebsiteSG inbound rule (HTTP, port 80, 0.0.0.0/0)

Create the IAM policy and role (S3-ARR-Role)

1. Open the **IAM console**, select **Policies** and click **Create policy**, then open the **JSON** tab.
2. Paste the contents of `bucket-permissions.json`, replacing `YOUR-BUCKET-ARN` with the ARN of your S3 bucket.
3. Review, create the policy and name it `S3-ARR-Policy`.
4. Go to **Roles** and click **Create role**. Choose **EC2** as the trusted entity and click **Next**.
5. Attach `S3-ARR-Policy`, name the role `S3-ARR-Role` and finish creating it.

PHASE 3

Launch the Red and Blue EC2 Instances

Launch the Red instance

1. In the **EC2 console** click **Launch instance**.
2. Name the instance `Red`, select an AMI and choose instance type **t2.micro**.
3. Under **Key pair**, choose **Proceed without a key pair**.

4. Under **Network settings**, select the `WebsiteSG` security group and the subnet in **us-east-1a**.
5. Expand **Advanced details** and set **IAM instance profile** to `S3-ARR-Role`.
6. In **User data**, paste the script from the `user-data-red` file. Replace `YOUR-BUCKET-NAME` with your real bucket name.
7. Click **Launch instance**.

The Red user data script installs and starts Apache, then copies the page files from S3 into both `/var/www/html/red` and the web root:

```
#!/bin/bash
yum update -y
yum install -y httpd
# below line starts httpd daemon
systemctl start httpd
# below line sets http daemon to start automatically at boot time
systemctl enable httpd
mkdir /var/www/html/red
# populate /red directory of server with web page
cd /var/www/html/red
aws s3 cp s3://YOUR-BUCKET-NAME/hw-red.css ./
aws s3 cp s3://YOUR-BUCKET-NAME/hw-red-py.css ./
aws s3 cp s3://YOUR-BUCKET-NAME/python.png ./
aws s3 cp s3://YOUR-BUCKET-NAME/apache.svg ./
aws s3 cp s3://YOUR-BUCKET-NAME/red-index.html ./index.html
# populate root of server with web page (replaces apache default page)
cd /var/www/html
aws s3 cp s3://YOUR-BUCKET-NAME/hw-red.css ./
aws s3 cp s3://YOUR-BUCKET-NAME/hw-red-py.css ./
aws s3 cp s3://YOUR-BUCKET-NAME/python.png ./
aws s3 cp s3://YOUR-BUCKET-NAME/apache.svg ./
aws s3 cp s3://YOUR-BUCKET-NAME/red-root-index.html ./index.html
# optional restart of httpd daemon
systemctl restart httpd
```

Figure 5: user-data-red script (replace YOUR-BUCKET-NAME before launching)

> **NOTE:** Apache is installed with `yum`, started and enabled with `systemctl`, and the files are pulled with `aws s3 cp`. This works only because the instance has the S3-ARR-Role attached.

Launch the Blue instance

Repeat every step above with these differences: name it `Blue`, place it in the subnet in **us-east-1b**, and use the `user-data-blue` script (with your bucket name filled in). Use the same `WebsiteSG` and `S3-ARR-Role`.

> **RESULT:** Once both instances are running you can open each instance's public IP in a browser and append `/red` or `/blue` to confirm the custom page loads.

PHASE 4

Create the Target Groups

1. In the **EC2 console**, under **Load Balancing**, open **Target Groups** and click **Create target group**.

Red target group

1. Target type: **Instances**. Name: `Red`. Leave the port as **80** and select your VPC.
2. Change the **Health check path** to `/red/index.html`.
3. Click **Next**, select the **Red** instance and click **Include as pending below**.

4. Create the target group.

Blue target group

1. Repeat the process with the name `Blue`.
2. Set the **Health check path** to `/blue/index.html`.
3. Select the **Blue** instance, click **Include as pending below** and create the target group.

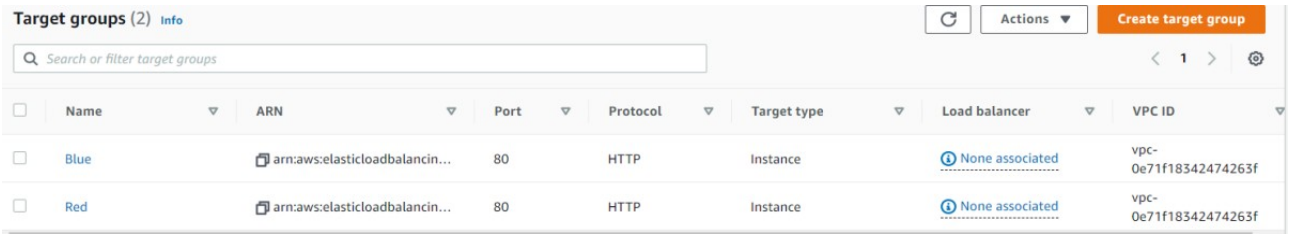


Figure 6: Red and Blue target groups (HTTP, port 80, target type Instance)

PHASE 5
Create the Application Load Balancer

1. In the EC2 console sidebar click **Load Balancers**, then **Create load balancer**.
2. Select **Application Load Balancer**.
3. Name it `LabLoadBalancer` and keep the scheme **Internet-facing**.
4. Under **Network mapping**, select both **us-east-1a** and **us-east-1b** so traffic can reach instances in both Availability Zones.
5. Under **Security groups**, select `WebsiteSG`.
6. Under **Listeners and routing**, set the listener to **HTTP : 80** with the default action **Forward to** the **Blue** target group.
7. Click **Create load balancer** and wait a few minutes until the state changes from *Provisioning* to *Active*.

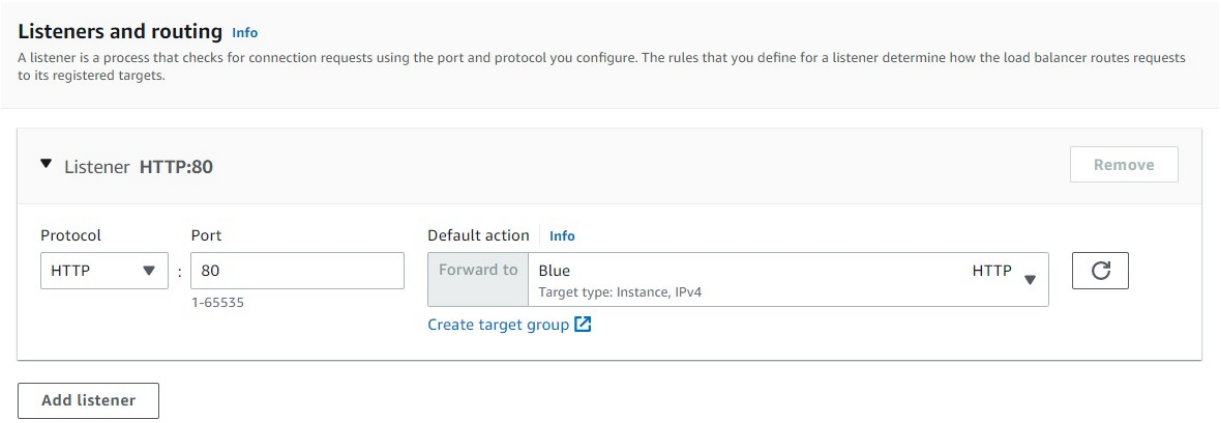


Figure 7: Listener HTTP:80 with default action forwarding to the Blue target group

PHASE 6
Configure Path-Based Routing Rules

1. When the ALB is active, open the **Listeners and rules** tab, tick the **HTTP:80** listener and choose **Manage rules > Add rule**.

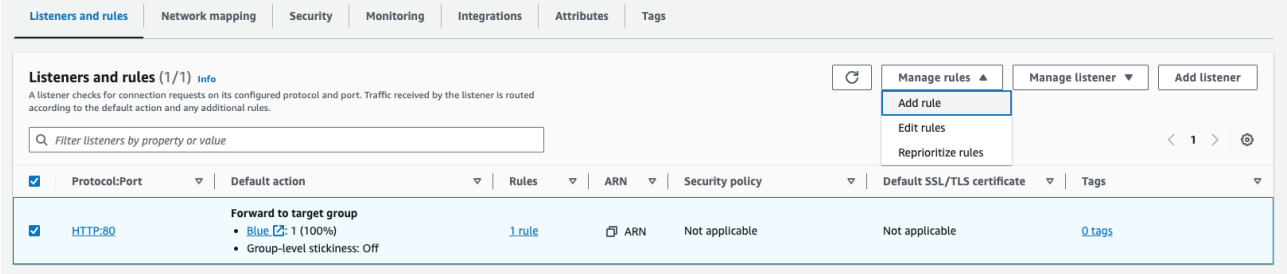


Figure 8: Manage rules menu on the HTTP:80 listener

Red routing rule

1. Ignore the name and tags step and click **Next**.
2. Condition: click **Add condition**, choose **Path** and enter `/red*`.
3. Action: choose **Forward to target groups** and select the **Red** target group.
4. Set **Priority** to **1** and save.

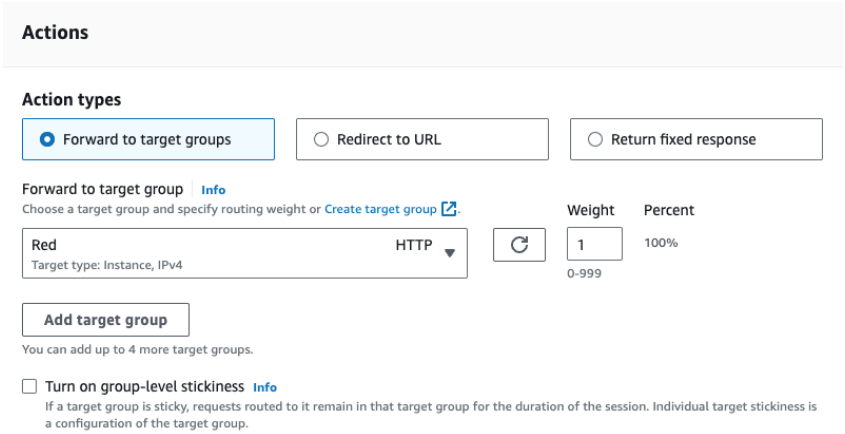


Figure 9: Rule action - Forward to target groups, Red (weight 1, 100%)

Blue routing rule

1. Add another rule with the **Path** condition `/blue*`.
2. Action: **Forward to target groups** and select the **Blue** target group.
3. Set **Priority** to **2** and save.

Resulting rules

The default action keeps forwarding unmatched traffic to the Blue target group. The listener should now show three rules:

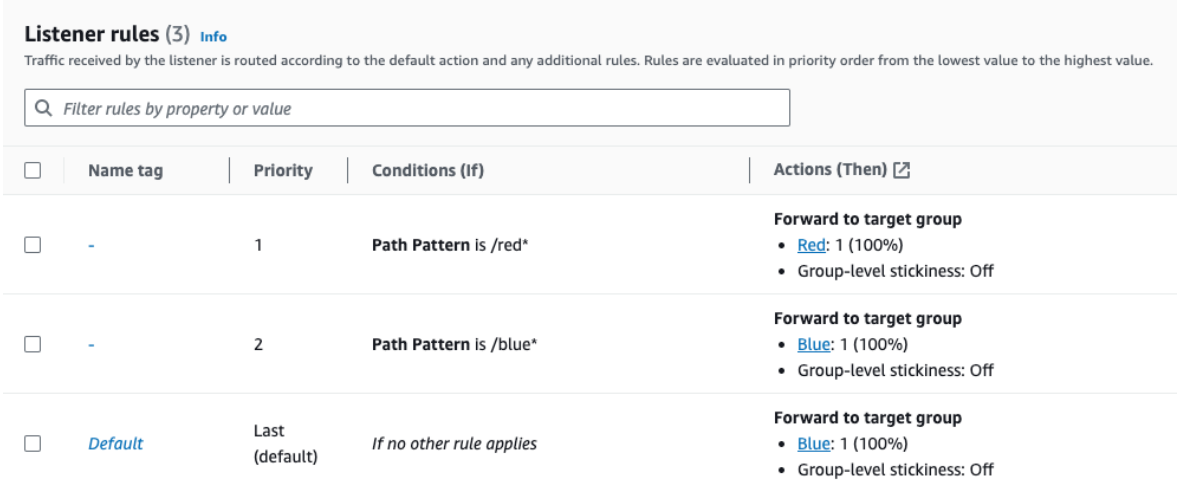


Figure 10: Path-based listener rules (Red = priority 1, Blue = priority 2, Default = Blue)

Test path-based routing

1. Open the load balancer and copy its **DNS name**.
2. Browse to `http://<ALB-DNS-name>/red`. The Red page should load.
3. Browse to `http://<ALB-DNS-name>/blue`. The Blue page should load.

> **RESULT:** Each colour is served by its own target group. If a page does not load, check that the target group shows the instance as **healthy** (the health check path must exist on the instance).

PHASE 7

Configure Host-Based Routing (Optional / Alternative)

Use this phase to switch from path-based to host-based routing. Only the listener rules change; the instances, target groups and ALB stay the same.

Remove the path-based rules

1. Go back to the HTTP:80 listener and choose **Manage rules > Edit rules**.
2. Select and delete both path-based rules (`/red*` and `/blue*`).

Red host rule

1. Add a rule with the condition **HTTP host header** and the value `red.yourdomainname.com` (use your own domain).
2. Action: **Forward to** the **Red** target group.
3. Priority: **1**.

Blue host rule

1. Add a rule with the condition **HTTP host header** and the value `blue.yourdomainname.com`.
2. Action: **Forward to** the **Blue** target group.
3. Priority: **2**.

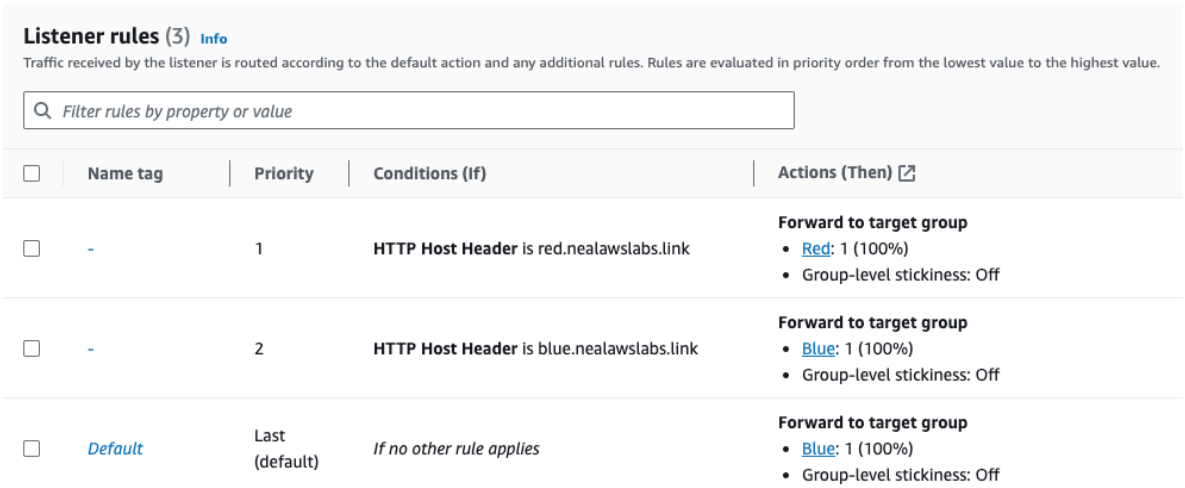


Figure 11: Host-header listener rules (example hostnames shown; use your own domain)

NOTE: The ALB matches the host header of each request. Requests only reach the right rule when the hostname in the request is the one in the rule. DNS configuration is outside the scope of this document, so you can verify the rules by sending the header directly, for example: `curl -H "Host: red.yourdomainname.com" http://<ALB-DNS-name>/`

4. Verification Checklist

Check	Expected result
S3 bucket	All web files uploaded (user data files and bucket-permissions.json excluded)
EC2 instances	Red in us-east-1a and Blue in us-east-1b are running with S3-ARR-Role attached
Target groups	Both Red and Blue show the instance as healthy
ALB state	Active, listener HTTP:80 with default action to Blue
Path rules	/red* goes to Red (priority 1); /blue* goes to Blue (priority 2)
Browser test	ALB-DNS/red shows the Red page; ALB-DNS/blue shows the Blue page

5. Clean Up Resources

To avoid unwanted charges, delete the resources when you have finished:

Resource	Chargeable?
Application Load Balancer	Yes
Amazon EC2 instances (Red, Blue)	Yes
Amazon S3 bucket	Yes
Target groups	No
Security group (WebsiteSG)	No

Suggested order: delete the load balancer first, then the target groups, terminate the EC2 instances, delete the security group, and finally empty and delete the S3 bucket. The IAM role and policy have no cost but can also be removed.

6. Key Takeaways

- An ALB can route on content (path or host header), which lets one load balancer front several applications.
- Listener rules are evaluated from the lowest priority number to the highest; the default rule catches everything else.
- Target group health checks must point to a path that really exists on each instance (/red/index.html, /blue/index.html).
- IAM roles let EC2 instances pull files from S3 securely without storing access keys.
- Deploying across two Availability Zones improves availability of the application.